CardDemo interest calculation: wiki pages and I/O contract

Wiki pages generated by the FlowX.AI legacy transformation agent from AWS CardDemo (open source, Apache-2.0). CONTRACT.md is the input/output contract for the build.

Pages included:

- `02-processes/process-interest-calculation.md`
- `10-acceptance-criteria/acceptance-criteria-interest-calculation.md`
- `01-data-model/project-data-model.md`
- `04-integrations/workflow-account-data-access.md`
- `04-integrations/workflow-reference-data-access.md`
- `04-integrations/workflow-transaction-data-access.md`
- `02-processes/process-end-of-day-batch-chain.md`
- `CONTRACT.md`

`02-processes/process-interest-calculation.md`

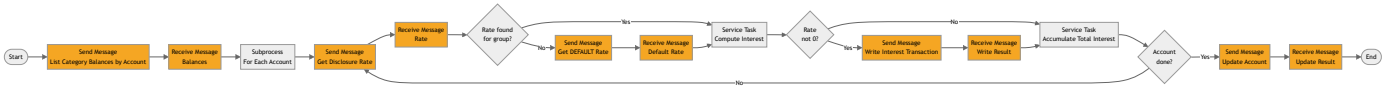
Process: interest-calculation

Property	Value
Type	Main Process (batch)
Legacy Source	Job `INTCALC` (`/app/jcl/INTCALC.jcl`, `PARM='2022071800'`), program `CBACT04C`, copybooks `CVTRA01Y`, `CVTRA02Y`, `CVACT01Y`, `CVACT03Y`, `CVTRA05Y`
Trigger	Called by end-of-day-batch-chain (monthly folder in Control-M)
Roles	Batch Operator (system)
Integration	workflow-reference-data-access, workflow-account-data-access, workflow-transaction-data-access
Acceptance Criteria	acceptance-criteria-interest-calculation
Mapping	P-15

Purpose

For each account/type/category balance, finds the disclosure-group interest rate, computes monthly interest, posts a system transaction (type 01, category 05) and updates the account balance, resetting cycle credit/debit totals.

Process Flow



Business Rules

#	Rule	Legacy Source
BR-01	Rate lookup key: `account.groupId + typeCode + categoryCode`	`CBACT04C`
BR-02	If not found (file status 23), retry with group `DEFAULT`	`CBACT04C`
BR-03	If rate ≠ 0: `monthlyInterest = (categoryBalance × rate) / 1200`	`CBACT04C` 1300-COMPUTE-INTEREST
BR-04	Interest transaction: type `01`, category `05`, source `System`, description `Int. for a/c <accountId>`, amount = interest	`CBACT04C`
BR-05	Transaction id = run date prefix (`PARM`, e.g. 2022071800) + running suffix. FlowX: `interestRunDate` Configuration Parameter + sequence	`CBACT04C`
BR-06	At each change of account: `account.currentBalance += totalInterest`; `currentCycleCredit = 0`; `currentCycleDebit = 0`; rewrite account	`CBACT04C`
BR-07	Fees (`1400-COMPUTE-FEES`) is an empty stub — no fees charged (finding F-12; improvement FI-15)	`CBACT04C`
BR-08	`monthlyInterest` is rounded to the nearest cent, halves away from zero (COBOL `COMPUTE ... ROUNDED`): 1000.40 × 18.00 / 1200 = 15.006 → 15.01; −15.006 → −15.01	`CBACT04C`
BR-09	Legacy opens the transaction output as OUTPUT (overwrites; finding F-07) — FlowX appends	`CBACT04C`

#	Rule	Legacy Source
BR-10	Payment (type 02) and Credit (type 03) categories have rate 0 in `discgrp.txt`, hence no interest	`discgrp.txt`

Data Mapping

Process start data: `interestRunDate` (Configuration Parameter default; override at start), `batchId`.

SEND `LIST_BALANCES_BY_ACCOUNT` → RECEIVE `balances[]` grouped per account. SEND `GET_DISCLOSURE_RATE` {groupId, typeCode, categoryCode} → RECEIVE {rate, usedFallback}. SEND `WRITE_INTEREST_TRANSACTION` → RECEIVE {transactionId}. SEND `UPDATE_ACCOUNT_AFTER_INTEREST` {accountId, totalInterest} → RECEIVE {status}.

Configuration Dependencies

Item	Bucket
interestRunDate	Configuration Parameter
divisor 1200	constant in script (12 months × 100)
DisclosureGroupId, TransactionTypeCode, TransactionCategoryCode	Enumeration / seed data

Acceptance Criteria: interest-calculation

Process: interest-calculation (P-15) · **UI Flow:** none (batch) · **Legacy:** `CBACT04C` / `INTCALC`

Scenarios

#	Scenario	Steps	Expected Result	Tags
1	Monthly interest	1. Category balance 1000.00, rate 18.00	Interest 15.00 (balance × rate / 1200)	@smoke @regression
2	Interest transaction	1. Run	System transaction type 01, category 05 created per category balance	@regression
3	Account update	1. Run	Account current balance increased by total interest; cycle credit/debit reset	@regression
4	Missing group	1. Account group has no rate row	Rate taken from `DEFAULT` group	@regression
5	Zero rate	1. Group `ZEROAPR`	No interest transaction	@regression
6	Run date	1. Provide `interestRunDate`	Transaction ids built from run date plus suffix	@regression
7	Fees	1. Run	No fee created (legacy stub F-12) {{DECISION: implement fees in FlowX or keep stub}}	@manual

▶ Gherkin

Legacy Comparison

Behaviour	Legacy	FlowX	Same?
Formula	bal × rate / 1200	same	Same
Fee computation	stub (F-12, FI-15)	stub / decision	Same
Rate lookup	DISCGRP read with DEFAULT fallback	W-05 reference-data-access	Same

Configuration Dependencies

`interestRunDate` (legacy PARM `2022071800`), `RuntimeSwitches.batchWindowOpen`.

Data Mapping Verification

TCATBAL → TransactionCategoryBalance, DISCGRP → DisclosureGroup (rate is a signed decimal, e.g. `0150{` = 15.00); verify interest is rounded to the nearest cent, halves away from zero (1000.40 × 18.00 / 1200 = 15.006 → 15.01).

Project Data Model

This page defines every **Project Data Type** of the AWS CardDemo FlowX application, derived from the legacy VSAM record layouts (`/app/cpy/*.cpy`), the optional-module IMS segments (`CIPAUSMY`, `CIPAUDTY`) and the DB2 DDL (`AUTHFRDS`, `TRANSACTION_TYPE`, `TRANSACTION_TYPE_CATEGORY`). The legacy-to-FlowX mapping is in the Transformation Matrix.

> All legacy VSAM / IMS / DB2 stores are replaced by **FlowX Persistence (App Data Store)**. Access always goes through an Integration Workflow (DATA_SOURCE node on the `carddemo-datastore` NOSQL_DB System). {{DECISION: Keep one NOSQL_DB System (`carddemo-datastore`) with one collection per Project Data Type, instead of separate systems per legacy store (VSAM / IMS / DB2).}}

Data Type Index

ID	Project Data Type	Legacy Source	Legacy Store / Record Size	Collection
DT-01	Account	`CVACT01Y`	VSAM KSDS ACCTDAT / 300	`accounts`
DT-02	Customer	`CVCUS01Y`	VSAM KSDS CUSTDAT / 500	`customers`
DT-03	Card	`CVACT02Y`	VSAM KSDS CARDDAT (+AIX CARDAIX) / 150	`cards`
DT-04	CardXref	`CVACT03Y`	VSAM KSDS CCXREF (+AIX CXACAIX) / 50	`card_xrefs`
DT-05	Transaction	`CVTRA05Y`	VSAM KSDS TRANSACT / 350	`transactions`
DT-06	TransactionCategoryBalance	`CVTRA01Y`	VSAM KSDS TCATBALF / 50	`tran_cat_balances`
DT-07	DisclosureGroup	`CVTRA02Y`	VSAM KSDS DISCGRP / 50	`disclosure_groups`
DT-08	TransactionType	`CVTRA03Y`, DB2 `TRANSACTION_TYPE`	VSAM TRANTYPE / 60, DB2	`transaction_types`
DT-09	TransactionCategory	`CVTRA04Y`, DB2 `TRANSACTION_TYPE_CATEGORY`	VSAM TRANCATG / 60, DB2	`transaction_categories`
DT-10	UserAccount	`CSUSR01Y`	VSAM KSDS USRSEC / 80	`user_accounts`
DT-11	DailyTransaction	`CVTRA06Y`	Sequential DALYTRAN / 350	`daily_transactions`
DT-12	RejectedTransaction	`CBTRN02C` (DALYREJS)	Sequential / 430	`rejected_transactions`

ID	Project Data Type	Legacy Source	Legacy Store / Record Size	Collection
DT-13	PendingAuthSummary	`CIPAUSMY` (IMS root)	IMS PAUTSUM0	`pending_auth_summaries`
DT-14	PendingAuthDetail	`CIPAUDTY` (IMS child)	IMS PAUTDTL1	`pending_auth_details`
DT-15	AuthFraudRecord	DB2 `CARDDEMO.AUTHFRDS`	DB2 table	`auth_fraud_records`
DT-16	SessionContext	`COCOM01Y` (COMMAREA)	CICS COMMAREA	process data (`session`)
DT-17	StatementData	`CBSTM03A/B` (in-memory)	Working storage + STMTFILE/HTMLFILE	`statements`
DT-18	BranchExportRecord	`CVEXPORT` / `CBEXPORT`	Sequential EXPFILE	`export_records`
DT-19	OptionalModuleToggles	CICS `INQUIRE PROGRAM` check (COMEN01C)	None	FeatureToggle Data Type

---

DT-01 Account

Legacy source: `/app/cpy/CVACT01Y.cpy` (300 bytes). Primary key `accountId`.

Attribute	Type	Required	Legacy Source	Description
accountId	String(11) numeric	Yes	`ACCT-ID` 9(11)	Account number, non-zero
activeStatus	Enumeration `AccountActiveStatus`	Yes	`ACCT-ACTIVE-STATUS` X(1)	Y / N
currentBalance	Number (2 dec)	Yes	`ACCT-CURR-BAL` S9(10)V99	Current balance
creditLimit	Number (2 dec)	Yes	`ACCT-CREDIT-LIMIT` S9(10)V99	Credit limit
cashCreditLimit	Number (2 dec)	Yes	`ACCT-CASH-CREDIT-LIMIT` S9(10)V99	Cash credit limit
openDate	Date (YYYY-MM-DD)	Yes	`ACCT-OPEN-DATE` X(10)	Account open date
expiryDate	Date	Yes	`ACCT-EXPIRAION-DATE` X(10) (legacy typo)	Account expiry date
reissueDate	Date	No	`ACCT-REISSUE-DATE` X(10)	Reissue date
currentCycleCredit	Number (2 dec)	Yes	`ACCT-CURR-CYC-CREDIT` S9(10)V99	Credits in current cycle
currentCycleDebit	Number (2 dec)	Yes	`ACCT-CURR-CYC-DEBIT` S9(10)V99	Debits in current cycle
addressZip	String(10)	No	`ACCT-ADDR-ZIP` X(10)	Billing ZIP
groupId	String(10)	Yes	`ACCT-GROUP-ID` X(10)	Disclosure Group id (A000000000 / DEFAULT / ZEROAPR)

Legacy FILLER (178 bytes) is dropped. Misspelling `EXPIRAION` is corrected (see Feature Improvements).

DT-02 Customer

Legacy source: `/app/cpy/CVCUS01Y.cpy` (500 bytes). Primary key `customerId`.

Attribute	Type	Required	Legacy Source	Description
customerId	String(9) numeric	Yes	`CUST-ID` 9(9)	Customer id
firstName	String(25)	Yes	`CUST-FIRST-NAME` X(25)	Alphabetic + spaces only
middleName	String(25)	No	`CUST-MIDDLE-NAME` X(25)	
lastName	String(25)	Yes	`CUST-LAST-NAME` X(25)	
addressLine1..3	String(50)	Line 1,2: Yes	`CUST-ADDR-LINE-1..3` X(50)	Street, line 2, city/other
addressState	String(2)	Yes	`CUST-ADDR-STATE-CD` X(2)	US state code
addressCountry	String(3)	Yes	`CUST-ADDR-COUNTRY-CD` X(3)	Country code
addressZip	String(10)	Yes	`CUST-ADDR-ZIP` X(10)	
phone1, phone2	String(15)	Phone1: Yes	`CUST-PHONE-NUM-1/2` X(15)	Format (AAA)BBB-CCCC
ssn	String(9) numeric (**sensitive**)	Yes	`CUST-SSN` 9(9)	Stored/displayed in clear in legacy; encrypt in FlowX
governmentId	String(20)	No	`CUST-GOVT-ISSUED-ID` X(20)	
dateOfBirth	Date	Yes	`CUST-DOB-YYYY-MM-DD` X(10)	Validated not in the future
eftAccountId	String(10)	No	`CUST-EFT-ACCOUNT-ID` X(10)	
primaryCardHolder	Boolean	Yes	`CUST-PRI-CARD-HOLDER-IND` X(1)	Y/N
ficoScore	Integer (300-850)	No	`CUST-FICO-CREDIT-SCORE` 9(3)	

DT-03 Card

Legacy source: `/app/cpy/CVACT02Y.cpy` (150 bytes). Primary key `cardNumber`; secondary index on `accountId` (replaces AIX `CARDAIX`).

Attribute	Type	Required	Legacy Source	Description
cardNumber	String(16) numeric	Yes	`CARD-NUM` X(16)	
accountId	String(11)	Yes	`CARD-ACCT-ID` 9(11)	FK to Account
cvv	String(3) (**sensitive**)	Yes	`CARD-CVV-CD` 9(3)	Clear text in legacy; tokenise
embossedName	String(50)	Yes	`CARD-EMBOSSED-NAME` X(50)	Alphabetic + spaces only
expiryDate	Date	Yes	`CARD-EXPIRAION-DATE` X(10)	Month 1-12, year validated

Attribute	Type	Required	Legacy Source	Description
activeStatus	Enumeration `CardActiveStatus`	Yes	`CARD-ACTIVE-STATUS` X(1)	Y / N

DT-04 CardXref

Legacy source: `/app/cpy/CVACT03Y.cpy` (50 bytes). Primary key `cardNumber`; secondary index on `accountId` (replaces AIX `CXACAIX`).

Attribute	Type	Required	Legacy Source	Description
cardNumber	String(16)	Yes	`XREF-CARD-NUM` X(16)	
customerId	String(9)	Yes	`XREF-CUST-ID` 9(9)	FK to Customer
accountId	String(11)	Yes	`XREF-ACCT-ID` 9(11)	FK to Account

DT-05 Transaction

Legacy source: `/app/cpy/CVTRA05Y.cpy` (350 bytes). Primary key `transactionId` (16 chars). Secondary access by `cardNumber` + processed timestamp (legacy AIX/`TRANIDX` — see {{TODO: confirm alternate index definition in /app/jcl/TRANIDX.jcl}}).

Attribute	Type	Required	Legacy Source	Description
transactionId	String(16)	Yes	`TRAN-ID` X(16)	Legacy: last record + 1 (collision risk); FlowX: sequence generator
typeCode	Enumeration `TransactionTypeCode`	Yes	`TRAN-TYPE-CD` X(2)	01-07
categoryCode	Enumeration `TransactionCategoryCode`	Yes	`TRAN-CAT-CD` 9(4)	
source	String(10)	Yes	`TRAN-SOURCE` X(10)	e.g. POS TERM, OPERATOR
description	String(100)	Yes	`TRAN-DESC` X(100)	
amount	Number (2 dec)	Yes	`TRAN-AMT` S9(9)V99	Format -99999999.99
merchantId	String(9) numeric	Yes	`TRAN-MERCHANT-ID` 9(9)	
merchantName	String(50)	Yes	`TRAN-MERCHANT-NAME` X(50)	
merchantCity	String(50)	Yes	`TRAN-MERCHANT-CITY` X(50)	
merchantZip	String(10)	Yes	`TRAN-MERCHANT-ZIP` X(10)	

Attribute	Type	Required	Legacy Source	Description
cardNumber	String(16)	Yes	`TRAN-CARD-NUM` X(16)	FK to Card
originalTimestamp	DateTime	Yes	`TRAN-ORIG-TS` X(26)	
processedTimestamp	DateTime	Yes	`TRAN-PROC-TS` X(26)	

DT-06 TransactionCategoryBalance

Legacy source `CVTRA01Y` (50 bytes). Composite key `accountId + typeCode + categoryCode`.

Attribute	Type	Required	Legacy Source	Description
accountId	String(11)	Yes	`TRANCAT-ACCT-ID` 9(11)	
typeCode	String(2)	Yes	`TRANCAT-TYPE-CD` X(2)	
categoryCode	String(4)	Yes	`TRANCAT-CD` 9(4)	
balance	Number (2 dec)	Yes	`TRAN-CAT-BAL` S9(9)V99	Running balance per category; basis of interest calculation

DT-07 DisclosureGroup

Legacy source `CVTRA02Y` (50 bytes). Composite key `groupId + typeCode + categoryCode`. Seed data: `/app/data/ASCII/discgrp.txt` (51 rows; groups `A000000000`, `DEFAULT`, `ZEROAPR`).

Attribute	Type	Required	Legacy Source	Description
groupId	String(10)	Yes	`DIS-ACCT-GROUP-ID` X(10)	
typeCode	String(2)	Yes	`DIS-TRAN-TYPE-CD` X(2)	
categoryCode	String(4)	Yes	`DIS-TRAN-CAT-CD` 9(4)	
interestRate	Number (2 dec)	Yes	`DIS-INT-RATE` S9(4)V99	Annual percent; seed file stores signed overpunch (`0150{` = 15.00) — convert at load

DT-08 TransactionType

Legacy sources `CVTRA03Y` / DB2 `TRANSACTION_TYPE(TR_TYPE CHAR(2) PK, TR_DESCRIPTION VARCHAR(50))` / `trantype.txt`.

Attribute	Type	Required	Legacy Source	Description
typeCode	String(2)	Yes	`TRAN-TYPE` / `TR_TYPE`	PK
description	String(50)	Yes	`TRAN-TYPE-DESC` / `TR_DESCRIPTION`	

DT-09 TransactionCategory

Legacy sources `CVTRA04Y` / DB2 `TRANSACTION_TYPE_CATEGORY` (PK `TRC_TYPE_CODE`+`TRC_TYPE_CATEGORY`, FK to TRANSACTION_TYPE ON DELETE RESTRICT) / `trancatg.txt` (18 rows).

Attribute	Type	Required	Legacy Source	Description
typeCode	String(2)	Yes	`TRAN-TYPE-CD` / `TRC_TYPE_CODE`	FK to TransactionType; delete restricted when categories exist
categoryCode	String(4)	Yes	`TRAN-CAT-CD` / `TRC_TYPE_CATEGORY`	
description	String(50)	Yes	`TRAN-CAT-TYPE-DESC` / `TRC_CAT_DATA`	

DT-10 UserAccount

Legacy source `/app/cpy/CSUSR01Y.cpy` (80 bytes), VSAM KSDS USRSEC key (8,0). Sample users in `DUSRSECJ.jcl`.

Attribute	Type	Required	Legacy Source	Description
userId	String(8)	Yes	`SEC-USR-ID` X(8)	Upper-cased on sign-on
firstName	String(20)	Yes	`SEC-USR-FNAME` X(20)	
lastName	String(20)	Yes	`SEC-USR-LNAME` X(20)	
password	Credential hash (**sensitive**)	Yes	`SEC-USR-PWD` X(8)	Plain text in legacy; replaced by FlowX IAM credential {{DECISION: Authentication delegated to the FlowX identity provider; `password` is not stored in the Project Data Type.}}
userType	Enumeration `UserType`	Yes	`SEC-USR-TYPE` X(1)	A = Admin, U = Regular

DT-11 DailyTransaction

Legacy `CVTRA06Y` (350 bytes), sequential file DALYTRAN (input of POSTTRAN). Same attributes as Transaction with prefix `DALYTRAN-` plus `cardNumber` used for xref lookup; additionally `batchId` and `receivedTimestamp` (new, FlowX-only).

DT-12 RejectedTransaction

Legacy: DALYREJS output of `CBTRN02C` = 350-byte transaction image + validation trailer.

Attribute	Type	Required	Legacy Source	Description
dailyTransaction	Object (DailyTransaction)	Yes	DALYREJS record body	Original record

Attribute	Type	Required	Legacy Source	Description
rejectReason	Enumeration `RejectReasonCode`	Yes	`WS-VALIDATION-FAIL-REASON` 9(4)	100-103
rejectReasonDescription	String(76)	Yes	`WS-VALIDATION-FAIL-REASON-DESC`	{{TODO: confirm trailer layout (4 + 76) in CBTRN02C working storage}}
reviewStatus	String	No	New	OPEN / RESOLVED (see Rejected Transaction view)

DT-13 PendingAuthSummary (optional authorization module)

Legacy `CIPAUSMY` (IMS root segment).

Attribute	Type	Required	Legacy Source	Description
accountId	String(11)	Yes	`PA-ACCT-ID` S9(11) COMP-3	Key
customerId	String(9)	Yes	`PA-CUST-ID` 9(9)	
authStatus	String(1)	No	`PA-AUTH-STATUS` X	
accountStatus	List of String(2) (max 5)	No	`PA-ACCOUNT-STATUS` X(2) x5	
creditLimit	Number	Yes	`PA-CREDIT-LIMIT` S9(9)V99 COMP-3	
cashLimit	Number	Yes	`PA-CASH-LIMIT`	
creditBalance	Number	Yes	`PA-CREDIT-BALANCE`	
cashBalance	Number	Yes	`PA-CASH-BALANCE`	
approvedCount / declinedCount	Integer	Yes	`PA-APPROVED-AUTH-CNT` / `PA-DECLINED-AUTH-CNT` S9(4) COMP	
approvedAmount / declinedAmount	Number	Yes	`PA-APPROVED-AUTH-AMT` / `PA-DECLINED-AUTH-AMT`	

DT-14 PendingAuthDetail (optional authorization module)

Legacy `CIPAUDTY` (IMS child segment). Key: `accountId` + `authDate` + `authTime` (stored inverted in IMS for descending order).

Attribute	Type	Required	Legacy Source	Description
accountId	String(11)	Yes	parent key	FK to PendingAuthSummary
authDate / authTime	Date / Time	Yes	`PA-AUTHORIZATION-KEY` (COMP-3)	
originalDate / originalTime	Date / Time	Yes	`PA-ORIG-DATE/TIME`	
cardNumber	String(16)	Yes	`PA-CARD-NUM` X(16)	
authType	String(4)	Yes	`PA-AUTH-TYPE` X(4)	
cardExpiry	String(4)	Yes	`PA-CARD-EXPIRY-DATE` X(4)	MMYY

Attribute	Type	Required	Legacy Source	Description
messageType / messageSource	String(6)	Yes	`PA-MESSAGE-TYPE`, `PA-MESSAGE-SOURCE`	
authIdCode	String(6)	Yes	`PA-AUTH-ID-CODE`	
responseCode	Enumeration `AuthResponseCode`	Yes	`PA-AUTH-RESP-CODE` X(2)	88 APPROVED = '00'
responseReason	Enumeration `AuthDeclineReasonCode`	Yes	`PA-AUTH-RESP-REASON` X(4)	
processingCode	Integer(6)	Yes	`PA-PROCESSING-CODE` 9(6)	
transactionAmount / approvedAmount	Number	Yes	`PA-TRANSACTION-AMT`, `PA-APPROVED-AMT` S9(10)V99	
merchantCategoryCode	String(4)	Yes	`PA-MERCHANT-CATAGORY-CODE`	
acquirerCountryCode	String(3)	No	`PA-ACQR-COUNTRY-CODE`	
posEntryMode	Integer(2)	No	`PA-POS-ENTRY-MODE` 9(2)	
merchantId / merchantName / merchantCity / merchantState / merchantZip	String	Yes	`PA-MERCHANT-ID` X(15), `-NAME` X(22), `-CITY` X(13), `-STATE` X(2), `-ZIP` X(9)	
transactionId	String(15)	Yes	`PA-TRANSACTION-ID`	
matchStatus	Enumeration `AuthMatchStatus`	Yes	`PA-MATCH-STATUS` X(1)	P/D/E/M
fraudFlag	Enumeration `AuthFraudFlag`	No	`PA-AUTH-FRAUD` X(1)	F/R
fraudReportDate	Date	No	`PA-FRAUD-RPT-DATE` X(8)	

DT-15 AuthFraudRecord (optional authorization module)

Legacy DB2 `CARDDEMO.AUTHFRDS` (written by `COPAUS2C`). PK `cardNumber + authTimestamp`. Attributes mirror PendingAuthDetail plus `accountId` (DECIMAL 11) and `customerId` (DECIMAL 9), plus fraud action (`F` confirm / `R` remove) and `fraudReportDate`. Legacy source column: each attribute maps 1:1 to the like-named `AUTHFRDS` column in `/app/app-authorization-ims-db2-mq/ddl/AUTHFRDS.ddl`.

DT-16 SessionContext (COMMAREA replacement)

Legacy `COCOM01Y` (CARDDEMO-COMMAREA). In FlowX this is **process data**, not a persisted type; it is passed from the Sign-on process to Menu and onwards via Call Activity input/output mapping.

Attribute	Type	Legacy Source	Description
fromTransactionId / fromProgram	String	`CDEMO-FROM-TRANID`, `CDEMO-FROM-PROGRAM`	Navigation origin (replaced by FlowX

Attribute	Type	Legacy Source	Description
			navigation)
toTransactionId / toProgram	String	`CDEMO-TO-TRANID`, `CDEMO-TO-PROGRAM`	Navigation target
userId	String(8)	`CDEMO-USER-ID`	
userType	Enumeration `UserType`	`CDEMO-USER-TYPE` (88 CDEMO-USRTYP-ADMIN/USER)	
programContext	Enumeration `ProgramContext`	`CDEMO-PGM-CONTEXT` (88 ENTER/REENTER)	Not needed in FlowX (stateful process)
customerId / accountId / cardNumber	String	`CDEMO-CUST-ID`, `CDEMO-ACCT-ID`, `CDEMO-CARD-NUM`	Selected record context
customerFirst/Middle/LastName	String	`CDEMO-CUST-FNAME/MNAME/LNAME`	
lastMap / lastMapset	String	`CDEMO-LAST-MAP`, `CDEMO-LAST-MAPSET`	Not needed

DT-17 StatementData

Built in memory by `CBSTM03A`/`CBSTM03B` (xref → customer → account → transactions, using 2-D table, ALTER/GO TO control flow). Output formats: fixed text `STMTFILE` and HTML `HTMLFILE`.

Attribute	Type	Legacy Source	Description
statementId	String	New	
accountId, customerId, cardNumber	String	xref	
customerName, customerAddress	String	Customer	
currentBalance, creditLimit	Number	Account	
ficoScore	Integer	Customer	Printed on statement
lines	List of Transaction summaries	Transaction file	id, description, amount
totalExpenditure	Number	computed	Sum of lines
statementDocumentRef	String	New	Reference to Document Plugin output

DT-18 BranchExportRecord

`CBEXPORT` writes a multi-record-type export (customer, account, xref, transaction, card) into a single sequential file (`CVEXPORT` layout); `CBIMPORT` reads it back and splits into target files with an error file. Record-type codes (from `CBEXPORT`): `C` = customer, `A` = account, `X` = card cross-reference, `T` = transaction, `D` = card. Common header: record type (1), timestamp (26), sequence number (COMP), branch id (4), region code (5), then a 460-byte data area redefined per type (total 500 bytes). Note the export copies `ssn` in clear (finding in Legacy Overview).

DT-19 OptionalModuleToggles (FeatureToggle Data Type)

Replaces the CICS `INQUIRE PROGRAM` check used by `COMEN01C` to hide menu options of non-installed optional modules. See Enumerations and the config bucket rules in Architecture Overview.

Attribute	Type	Default	Description
pendingAuthorizationEnabled	Boolean	false	Menu option 11 (COPAUS0C)
transactionTypeMaintenanceEnabled	Boolean	false	Admin options 5 and 6 (COTRTLIC / COTRTUPC)
mqInquiryEnabled	Boolean	false	MQ date/account inquiry services

Relationships

See Data Model Diagram. Key referential rules (legacy enforced in code, not in VSAM):

- Card → Account via `accountId`; Card → Customer via CardXref.
- Transaction → Card; TransactionCategoryBalance → Account + TransactionCategory.
- TransactionCategory → TransactionType (DB2 FK ON DELETE RESTRICT; VSAM had none).
- Account.groupId → DisclosureGroup.groupId (fallback `DEFAULT` when no row for the group).

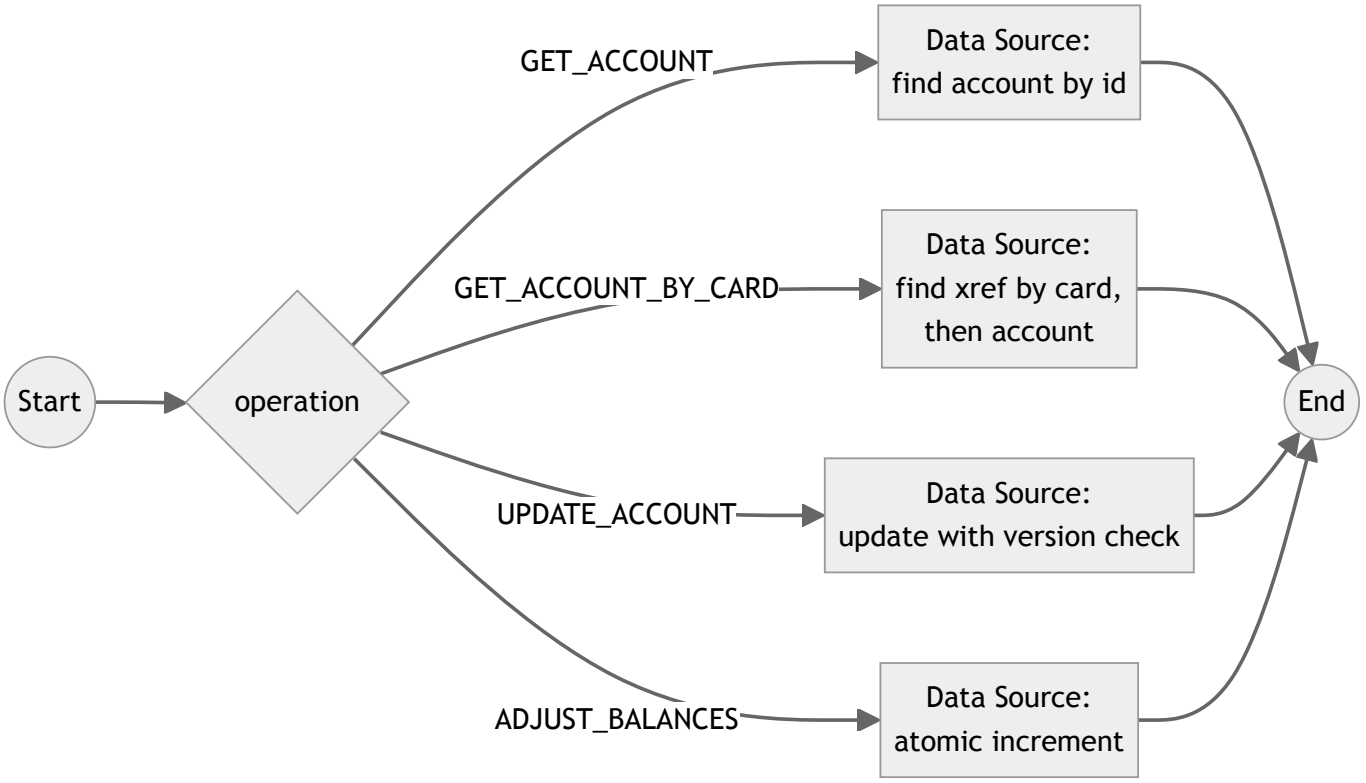
Workflow: account-data-access

Metadata

Field	Value
System	carddemo-datastore (NOSQL_DB)
Trigger Source	Process (MESSAGE_EVENT_SEND with StartIntegrationWorkflowHandler)
Trigger Processes	P-03, P-04, P-10, P-11, P-14, P-15, P-17, P-22, P-28
Correlation	`processInstanceId`
Legacy Source	`READ/REWRITE ACCTDAT`, `CARDXREF` alternate-index reads in `COACTVWC`, `COACTUPC`, `COBIL00C`, `COTRN02C`, `CBTRN02C`, `CBACT04C`, `COACCT01`
Mapping	W-01

{{DECISION: One workflow per Project Data Type with an `operation` input and a Condition node that routes to the matching DATA_SOURCE node, to keep the number of workflows manageable; split into one workflow per operation if independent versioning is preferred.}}

Workflow Flow



Nodes and Operations

Operation	Node	Data Source operation	Input	Output
GET_ACCOUNT	find account	find one in `accounts` by `accountId`	`accountId`	`account`, `found`
GET_ACCOUNT_BY_CARD	find xref, find account	find one in `cardXrefs` by `cardNumber`, then `accounts`	`cardNumber`	`account`, `accountId`, `customerId`, `found`
UPDATE_ACCOUNT	update account	update `accounts` where `accountId` and `version` = `expectedVersion`	`account`, `expectedVersion`	`updated`, `conflict`
ADJUST_BALANCES	adjust	increment `currentBalance`, `cycleCredit`, `cycleDebit`	`accountId`, deltas	`account`, `status`

Data Mappings

SEND payload (process → workflow): `operation`, `accountId` / `cardNumber`, `account` (for update), `expectedVersion`, deltas. RECEIVE payload (workflow → process, written to `accountResult`): `found`, `conflict`, `account.*`, `status`, `message`.

Direction	Source	Target
Process → Send	`${selectedAccountId}`	`jsonDTOMessage.accountId`
Workflow → API	`workflow.accountId`	filter `accountId`
API → Workflow	`result.document`	`workflow.account`
End → Receive	`END.payload.account`	`accountResult.account`

Error Handling

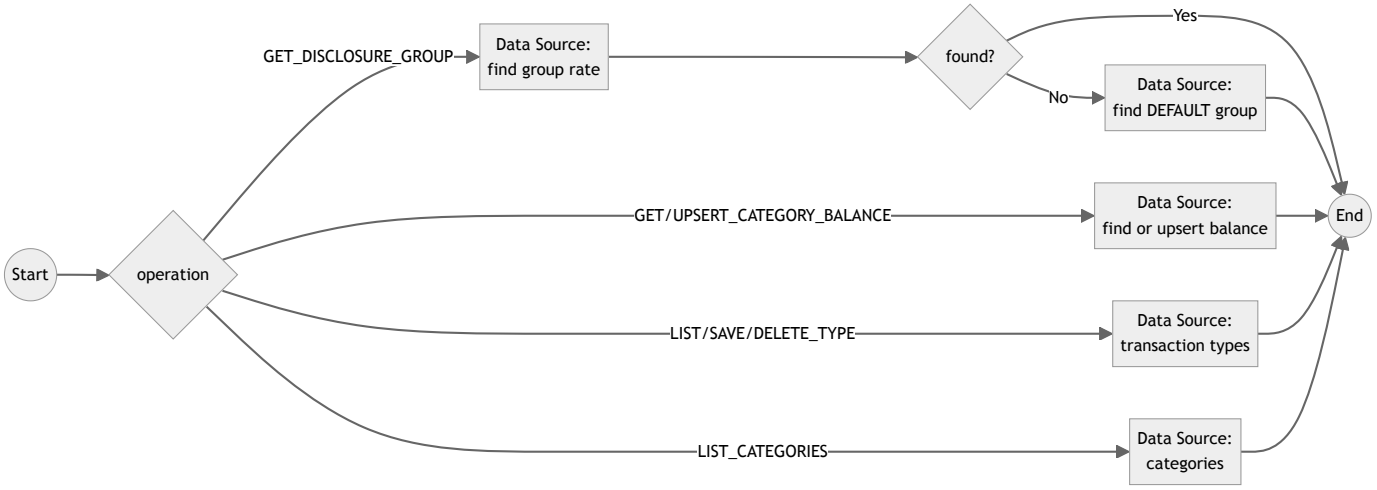
Error	Handling	Outcome
Not found (legacy `RESP NOTFND`)	`found = false`	Process shows the "not found" message
Version conflict	`conflict = true`	Process shows "Record changed by some one else. Please review"
Store timeout	retry 3 times with backoff	then error status to process, no data change

Workflow: reference-data-access

Metadata

Field	Value
System	carddemo-datastore (NOSQL_DB)
Trigger Source	Process (MESSAGE_EVENT_SEND with StartIntegrationWorkflowHandler)
Trigger Processes	P-14, P-15, P-18, P-21, P-25, P-26
Legacy Source	`TCATBALF`, `DISCGRP`, `TRANTYPE`, `TRANCATG` (VSAM) and DB2 `TRANSACTION_TYPE`, `TRANSACTION_TYPE_CATEGORY`; programs `CBTRN02C`, `CBACT04C`, `CBTRN03C`, `COTRTLIC`, `COTRTUPC`, `COBTUPDT`
Mapping	W-05

Workflow Flow



Nodes and Operations

Operation	Data Source operation	Input	Output
GET_DISCLOSURE_GROUP	find `disclosureGroups` by group, type, category; if missing, find with group `DEFAULT`	`groupId`, `typeCode`, `categoryCode`	`interestRate`, `usedDefault`
GET_CATEGORY_BALANCE / UPSERT_CATEGORY_BALANCE	find / upsert `categoryBalances`	`accountId`, `typeCode`, `categoryCode`, `delta`	`balance`
LIST_CATEGORY_BALANCES	query by `accountId`	`accountId`	`balances[]`

Operation	Data Source operation	Input	Output
LIST_TYPES	query `transactionTypes` with filters, paged	`typeFilter`, `descriptionFilter`, `cursor`	`types[]`
SAVE_TYPE / DELETE_TYPE	upsert / delete; delete blocked if categories exist	`type`	`status`
LIST_CATEGORIES	query `transactionCategories` by `typeCode`	`typeCode`	`categories[]`
GET_TYPE_DESCRIPTION	lookup description (report)	`typeCode`, `categoryCode`	`typeDescription`, `categoryDescription`

Data Mappings

RECEIVE (`referenceResult`): `interestRate`, `usedDefault`, `balance`, `balances[]`, `types[]`, `categories[]`, `status`, `message`. Interest rate is stored as a decimal percentage (legacy encoded value such as `0150{` = 15.00 is converted at load time, see enumerations).

Error Handling

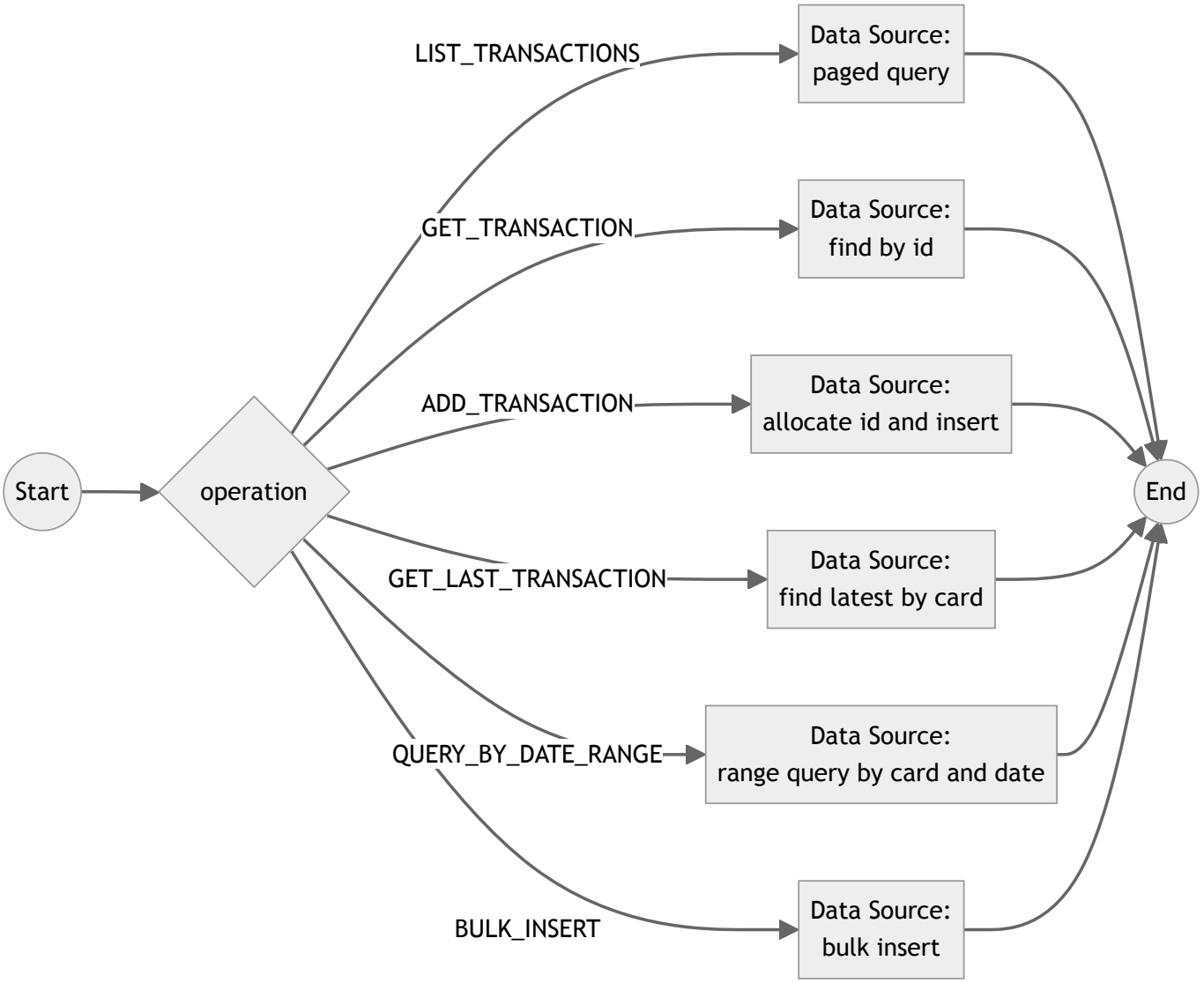
Missing disclosure group → DEFAULT fallback (BR in P-15); delete with children → status `RESTRICTED`; timeout → retry 3 times.

Workflow: transaction-data-access

Metadata

Field	Value
System	carddemo-datastore (NOSQL_DB)
Trigger Source	Process (MESSAGE_EVENT_SEND with StartIntegrationWorkflowHandler)
Trigger Processes	P-08, P-09, P-10, P-11, P-14, P-15, P-16, P-17, P-18
Legacy Source	`COTRN00C` – `COTRN02C`, `COBIL00C` (`TRANSACT` browse/read/write), `CBTRN02C`, `CBACT04C`, `CBTRN03C`, `CBSTM03A`, `COMBTRAN.jcl`
Mapping	W-04

Workflow Flow



Nodes and Operations

Operation	Data Source operation	Input	Output
LIST_TRANSACTIONS	query `transactions` ordered by id, start at `startTransactionId`, page of 10	`startTransactionId`, `direction`, `pageSize`	`transactions[]`, `hasNext`, `hasPrevious`
GET_TRANSACTION	find by `transactionId`	`transactionId`	`transaction`, `found`
ADD_TRANSACTION	insert; the new id is allocated by a sequence (replaces "last record + 1")	`transaction`	`transactionId`
GET_LAST_TRANSACTION	latest transaction of a card (PF5 copy-last)	`cardNumber`	`transaction`
QUERY_BY_DATE_RANGE	transactions between `startDate` and `endDate` (report, statement)	`cardNumber/accountId`, dates, `cursor`	`transactions[]`, `nextCursor`
BULK_INSERT	insert many (posting, interest, load)	`transactions[]`	counts

Data Mappings

RECEIVE (`transactionResult`): `transactions[]` / `transaction.*`, `found`, `transactionId`, counts, `nextCursor`. The id sequence replaces the `STARTBR/READPREV/ENDBR` last-record logic (F-10 duplicate id collision).

Error Handling

Not found → `found = false`; duplicate id → retry with a new sequence value; timeout → retry 3 times then error status; bulk insert failures are returned per record.

Process: end-of-day-batch-chain

Property	Value
Type	Main Process (orchestrator for batch Call Activities)
Legacy Source	Control-M folders (`/app/scheduler/CardDemo.controlm`), CA-7 definition (`/app/scheduler/CardDemo.ca7`, schedule 000652), jobs `CLOSEFIL`, `TRANBKP`, `WAITSTEP`, `OPENFIL`, `POSTTRAN`, `INTCALC`, `COMBTRAN`, `CREASTMT`, `TRANREPT`, `DISCGRP`; assembler `MVSWAIT` / COBOL `COBSWAIT`
Trigger	Scheduled Trigger (daily / weekly / monthly) — see Trigger Inventory
Roles	Batch Operator (system)
Acceptance Criteria	acceptance-criteria-end-of-day-batch-chain
Mapping	P-19

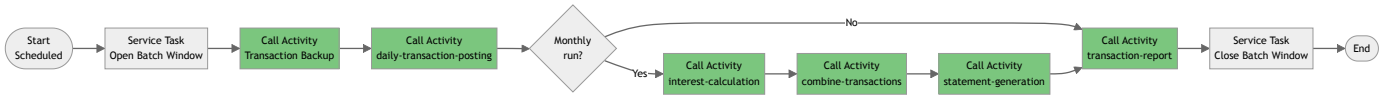
Purpose

Coordinates the nightly/monthly batch cycle that in the mainframe is: close CICS files (`CLOSEFIL`) → post transactions → interest → combine → statements → reports → reopen CICS files (`OPENFIL`).

{{DECISION: In FlowX there are no CICS files to close/open. `CLOSEFIL`, `OPENFIL` and `WAITSTEP` (pure wait) are **dropped** from the technical flow and replaced by a processing-window flag (`batchWindowOpen`) that blocks online data-changing processes during the run.}}

{{TODO: confirm with business whether online access must be blocked during the batch run}}

Process Flow



Legacy Job Mapping

Legacy Job	Program	Control-M Folder	FlowX Node
CLOSEFIL	IEFBR14/CICS close	DAILY, WEEKLY(DISCGRP), MONTHLY	Open Batch Window
TRANBKP	IDCAMS REPRO	DAILY-TransactionBackup	Transaction Backup (Call Activity: export `transactions` snapshot)
WAITSTEP	`COBSWAIT` / `MVSWAIT`	DAILY, WEEKLY, MONTHLY	dropped
OPENFIL	CICS open	DAILY, WEEKLY, MONTHLY	Close Batch Window
POSTTRAN	`CBTRN02C`	(chain)	daily-transaction-posting

Legacy Job	Program	Control-M Folder	FlowX Node
INTCALC	`CBACT04C`	MONTHLY-InterestCalculation	interest-calculation
COMBTRAN	SORT/REPRO	MONTHLY-InterestCalculation	combine-transactions
CREASTMT	`CBSTM03A/B`	(chain)	statement-generation
TRANREPT	`CBTRN03C`	(chain)	transaction-report
DISCGRP	IDCAMS REPRO	WEEKLY-DisclosureGroupsRefresh	master-data-load (reference refresh)
MNTTRDB2	`COBTUPDT`	WEEKLY-TransactionTypesDBRefresh	transaction-type-batch-maintenance
CBPAUP0J	`CBPAUP0C`	triggered from CLOSEFIL (CA-7 SCHID 030)	expired-authorization-purge

Scheduler Facts

- Control-M folders (APPLICATION `CARD-DEMO`, RUN_AS `xuemingh`, TIMETO 23:00): DAILY-TransactionBackup, WEEKLY-TransactionTypesDBRefresh, WEEKLY-DisclosureGroupsRefresh, MONTHLY-InterestCalculation.
- `TRANEXTR` is listed under the WEEKLY-DisclosureGroupsRefresh parent folder (misplacement, finding F-16).
- CA-7: system `CARDDEMO`, JCLLIB `&CARDDEMOPRODJCL`, schedule 000652; `CLOSEFIL` triggers `CBPAUP0J` (SCHID=030).
- Cron expressions are not defined in the extracted definitions: {{TODO: obtain run days/times (the Control-M XML shows all months enabled and TIMETO=23:00 only)}}.

Business Rules

#	Rule	Legacy Source
BR-01	Order: posting before interest, interest before combine, combine before statements, statements before reports	Control-M / README job chain
BR-02	A failing step stops the chain (`COND=(0,NE)` on later steps)	`CREASTMT.JCL`, others
BR-03	Posting with rejects (`RETURN-CODE 4`) does not stop the chain	`CBTRN02C`
BR-04	Run window ends by 23:00 (`TIMETO`)	Control-M

Data Mapping

Process start data: `batchId`, `businessDate`, `runType` (DAILY / WEEKLY / MONTHLY). Call Activity INPUT: `batchId`, `businessDate`, `interestRunDate`. OUTPUT: `status`, `processedCount`, `rejectedCount`.

Configuration Dependencies

`schedulerEndTime`, `interestRunDate`, `reportStartDate`, `reportEndDate`, `batchWindowOpen` (runtime flag → FeatureToggle Data Type `OptionalModuleToggles` extension {{DECISION: add `batchWindowOpen` to a

separate `RuntimeSwitches` FeatureToggle}}).

Candidate I/O contract

What a migrated implementation must produce so the scorer can compare it with the oracle. The contract fixes **formats only**. It says nothing about behaviour, so it may be handed to a builder without leaking the answers.

Inputs (given per run)

The same files the COBOL chain reads, one record per line, fixed width, layouts as in the copybooks:

File	Copybook	Record
`dailytran.dat`	CVTRA06Y	350
`acctdata.dat`	CVACT01Y	300
`cardxref.dat`	CVACT03Y	50
`tcatbal.dat`	CVTRA01Y	50
`discgrp.dat`	CVTRA02Y	50
`custdata.dat`, `carddata.dat`, `trantype.dat`, `trancatg.dat`	CUSTREC, CVACT02Y, CVTRA03Y, CVTRA04Y	500, 150, 60, 60

Run parameters: business date `2022-07-18`, INTCALC parameter `2022071800`.

Step isolation. A candidate for INTCALC receives the oracle's POSTTRAN outputs as its account and category-balance inputs, so an INTCALC result is never penalised for a POSTTRAN error.

Outputs

One JSON file per output: a list of objects, in any order. Field names and types are fixed. Strings may omit trailing spaces. Amounts are JSON strings or numbers holding at most two decimals (`"14.56"`, `-500`).

File	Step	Fields
`posttran_transact.json`	POSTTRAN	transaction fields (below) for every posted transaction
`posttran_dalyrejs.json`	POSTTRAN	transaction fields + `reason` (int) + `reason_desc` for every rejected transaction
`posttran_acctdata.json`	POSTTRAN	account fields for every account, after posting
`posttran_tcatbal.json`	POSTTRAN	`acct_id, type_cd, cat_cd, bal` for every category balance, after posting
`intcalc_systran.json`	INTCALC	transaction fields for every interest transaction written
`intcalc_acctdata.json`	INTCALC	account fields for every account, after interest

- Transaction fields: `tran_id, type_cd, cat_cd (int), source, desc, amt, merchant_id (int), merchant_name, merchant_city, merchant_zip, card_num, orig_ts, proc_ts`
- Account fields: `acct_id (int), active_status, curr_bal, credit_limit, cash_credit_limit, open_date, expiration_date, reissue_date, curr_cyc_credit, curr_cyc_debit, addr_zip, group_id`

A run may instead leave the oracle's fixed-width dumps (`<name>.txt`). The scorer reads either form.

Optional `flags.json`: the uncertainties the pipeline declared **before** scoring, as `[{"rule": "R-INT-TRUNC", "source": "<artefact path and line>"}]` or `[{"output": "...", "key": "...", "source": "..."}]`. See PROTOCOL.md §6 for what counts as a flag.